

Outpainting Audio by Reusing What the Model Already Knows: A Minimal Full-Finetune Recipe for Latent Rectified-Flow Audio Diffusion

Francesco Colonnese
github.com/mrcolo/stems-loop

Abstract

Audio outpainting — continuing a recording past its end — is usually framed as a new capability requiring new machinery: extra input channels, adapter branches, or a separately trained continuation head. We observe that a modern latent audio diffusion model already contains everything the task needs. Stable Audio 3 Medium is trained with inpainting conditioning, taking a binary mask and a masked latent as local-additive conditions, and its mask distribution already includes a causal mode that retains a random-length prefix. Outpainting is that mode. Consequently a finetune for outpainting can change no architecture at all: no new channels, no adapters, no surgery on the input projection. Only the transformer weights move, and step zero is exactly the pretrained model rather than a perturbed one.

We describe a complete recipe built on this observation, targeting a single 24 GB consumer GPU. It trains all 1.453 B transformer parameters, streams excerpts directly from multi-hour source audio with no preprocessing stage, and runs the frozen autoencoder online so that no latent cache is ever materialised. We detail the memory budget that makes full finetuning fit, the masked rectified-flow objective, and a frozen-slice evaluation protocol that makes the learning signal legible — ordinary training loss in this setting is dominated by timestep variance and is uninformative. The entire method is three files and roughly 250 lines.

1 Introduction

Generative audio models have converged on latent diffusion: a high-compression autoencoder maps waveforms to a short sequence of continuous vectors, and a diffusion transformer (DiT) is trained in that space. The recipe is now mature enough that the interesting questions have shifted from how to generate to how to control, and one of the most practically useful controls is continuation: given the first τ seconds of a recording, produce what plausibly follows.

The standard reflex is to treat this as an architectural problem. Image inpainting models concatenate a mask and a masked image to the noisy latent, widening the input projection from C to $2C+1$ channels and zero-initialising the new columns. The same construc-

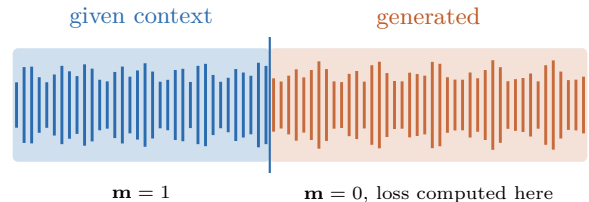


Figure 1: The task. A random-length prefix of a latent sequence is retained as context; the model generates the remainder. This is exactly the causal member of the mask family the base model was pretrained with, so no architectural change is required. The training loss is taken only over the generated region.

tion transfers to audio and works. But it is unnecessary when the base model was already trained with masked conditioning.

Contributions. (i) We identify that outpainting is a subset of the inpainting conditioning already present in Stable Audio 3, and show that restricting the pretrained mask distribution toward its causal mode is sufficient to specialise the model for continuation (§4.1). (ii) We give a memory budget under which all 1.453 B transformer parameters can be finetuned on one 24 GB GPU while a 0.852 B autoencoder runs online in the same step (§6). (iii) We describe a data path that requires no preprocessing, no manifest and no latent cache, reading random excerpts directly out of multi-hour source files (§5). (iv) We argue that per-step training loss is the wrong instrument for this objective and propose a frozen evaluation slice that makes progress measurable (§8).

2 Related Work

Latent audio diffusion. Encoding waveforms into a compact latent sequence before diffusion is now standard, and the compression ratio has risen sharply: the autoencoder we build on maps 4096 waveform samples to a single 256-dimensional latent frame, a 10.77 Hz representation of 44.1 kHz stereo. High compression is what makes minute-scale generation tractable, and it is also what makes online encoding expensive (§7).

Masked generative modelling. Inpainting by channel-concatenating a mask and a masked signal originates in image diffusion and has been carried into audio. The variant we exploit differs in that the masked conditioning is injected as a local additive term rather than by widening the input projection, and — critically — is present at pretraining rather than bolted on afterwards.

Flow matching and rectified flow. Rectified flow replaces the variance-preserving diffusion path with a straight interpolation between data and noise, and predicts the constant velocity along it. Its simplicity and its compatibility with few-step distilled samplers have made it the default for recent audio models.

Full versus parameter-efficient finetuning. LoRA and adapters dominate finetuning practice because they fit on small hardware. We take the opposite position for this task: when the target domain differs from the pretraining distribution in timbre and production style rather than in semantics, the low-rank constraint is a real limitation, and a 1.45 B model can be fully finetuned on a consumer card given the right precision and optimiser-state choices.

3 Preliminaries

3.1 Latent representation

Let $\mathbf{x} \in \mathbb{R}^{2 \times N}$ be stereo audio at 44.1 kHz. A frozen autoencoder $\mathcal{E}$ maps it to

$$\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{256 \times T}, \quad T = \lceil N/4096 \rceil, \quad (1)$$

a 256-channel sequence at 10.77 frames per second. The encoder is a patched transformer: waveform samples are first grouped into patches of 256, then a 12-layer transformer with stride 16 produces the latent. The decoder mirrors it. Together they account for 0.852 B parameters and remain frozen throughout.

3.2 Rectified-flow objective

The generative model is a DiT v_θ trained with a rectified-flow (rf_denoiser) objective. For $t \in [0, 1]$ and $\epsilon \sim \mathcal{N}(0, I)$,

$$\mathbf{z}_t = (1-t)\mathbf{z} + t\epsilon, \quad (2)$$

$$\mathbf{v}^* = \epsilon - \mathbf{z}, \quad (3)$$

so that $t=0$ is data and $t=1$ is noise, and the target velocity is constant along each path. The network is conditioned on t , on a text embedding, and on a scalar duration embedding.

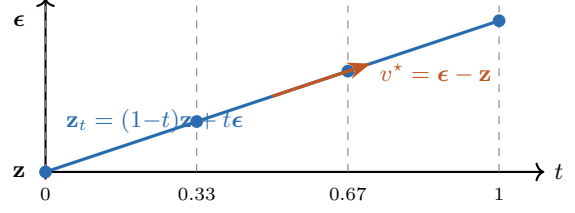


Figure 2: Rectified flow. Data and noise are joined by a straight line, and the network regresses the constant velocity along it. Because the path is linear, the velocity target does not depend on t — but the difficulty of predicting it does, which is the source of the variance discussed in §8.

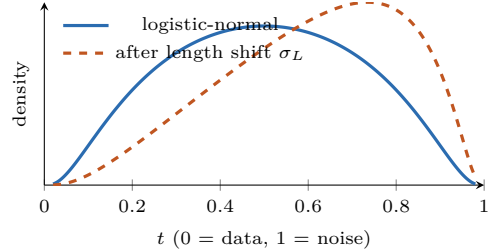


Figure 3: Timestep density. Mass concentrates in the middle of the schedule where the prediction problem is hardest, and the length-dependent shift moves it toward higher noise as the sequence grows. We inherit both from pretraining rather than substituting a uniform sampler.

3.3 Timestep sampling

Timesteps are drawn from a truncated logistic-normal distribution, reflected, and then warped by a sequence-length-dependent shift σ_L that moves probability mass toward higher noise for longer sequences:

$$t \sim \sigma_L(1 - \text{TLN}(0.075, 1)). \quad (4)$$

We reuse the pretraining sampler unchanged. Matching it matters: a finetune that samples timesteps differently from pretraining spends its gradient budget on a different part of the schedule than the one the model was optimised for.

4 Method

4.1 Outpainting as a restriction of pretrained masking

The base model consumes two extra conditioning tensors alongside the noisy latent: a binary keep-mask $\mathbf{m} \in \{0, 1\}^{1 \times T}$ and the masked latent $\mathbf{z} \odot \mathbf{m}$. These enter the transformer as a local additive term of dimension $256+1$, added to the token embeddings rather than concatenated to the input channels.

During pretraining, $\mathbf{m}$ is drawn from a family of mask types. Three matter here:

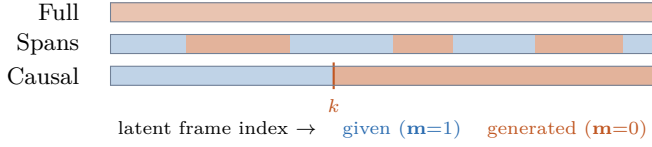


Figure 4: The pretrained mask family. Stable Audio 3 is pretrained with mask-type probabilities $[0.1, 0.8, 0.1]$ over [Spans, Full, Causal]. Outpainting is already in the support of this distribution — it is simply rare. Our finetune reweights to $[0, 0.1, 0.9]$.

Full $\mathbf{m} = 0$ everywhere. Pure unconditional generation.

Spans a set of non-overlapping interior spans is zeroed. General inpainting.

Causal a prefix length $k \sim \mathcal{U}\{0, \dots, T\}$ is sampled and $\mathbf{m}_i = \mathbf{1}[i < k]$. This is outpainting.

Because Causal is already in the support of the pre-training distribution, the finetune is a reweighting rather than an extension. We set the mask-type probabilities to $[0, 0.1, 0.9]$, concentrating gradient on continuation while retaining a tenth of each batch at Full. That residual matters: training exclusively on causal masks steadily degrades the model’s unconditional behaviour, which is also the behaviour the Full branch of classifier-free guidance relies on at sampling time.

4.2 Masked objective

The loss is taken only where the model is being asked to generate:

$$\mathcal{L}(\theta) = \frac{\| (v_\theta(\mathbf{z}_t, t, c, \mathbf{m}, \mathbf{z} \odot \mathbf{m}) - v^*) \odot (1 - \mathbf{m}) \|_2^2}{256 \cdot \sum_i (1 - \mathbf{m}_i)}. \quad (5)$$

Including the context region would let the model reduce $\mathcal{L}$ by reproducing input it was already handed, which is not the task and dilutes the gradient in proportion to the mask ratio. The normalisation by the number of generated elements keeps the loss scale independent of how much of the sequence happens to be masked in a given batch.

4.3 Conditioning is constant, so compute it once

The model is conditioned on a text embedding from a T5Gemma encoder (256 tokens, 768-dimensional) and on a scalar duration embedding. For a single-domain finetune both the prompt and the excerpt length are fixed for the entire run, so the conditioning tensor is a constant. We evaluate the text encoder once at startup, cache the resulting tensor, and release the encoder’s 1.2 GB before the first training step. Classifier-free-guidance dropout is applied downstream by zeroing the cached tensor and therefore needs no encoder.

Algorithm 1 One training step

- 1: $\mathbf{x} \leftarrow \text{RandomExcerpts}(\text{source}, B, \tau)$ ▷ 31 ms, on workers
 - 2: $\mathbf{z} \leftarrow \mathcal{E}(\mathbf{x})$ ▷ online, bf16, no grad
 - 3: $\mathbf{m} \leftarrow \text{CausalMask}(B, T; p_{\text{full}})$
 - 4: $t \leftarrow \sigma_L(1 - \text{TLN}())$
 - 5: $\epsilon \sim \mathcal{N}(0, I)$
 - 6: $\mathbf{z}_t \leftarrow (1 - t)\mathbf{z} + t\epsilon; \quad v^* \leftarrow \epsilon - \mathbf{z}$
 - 7: $\hat{v} \leftarrow v_\theta(\mathbf{z}_t, t, c, \mathbf{m}, \mathbf{z} \odot \mathbf{m})$ ▷ cfg dropout 0.1
 - 8: $\mathcal{L} \leftarrow \|(\hat{v} - v^*) \odot (1 - \mathbf{m})\|_2^2 / \sum_i (1 - \mathbf{m}_i)$
 - 9: backward; clip to 1.0; AdamW8bit step
-

5 Data path

Conventional latent-diffusion training precomputes latents into a cache. Stability’s own configuration does this (pre_encoded: true). We deliberately do not, and the data path is correspondingly trivial: on each item we open the source file, seek to a uniformly random frame offset, and read exactly the excerpt we need.

This has three consequences worth stating. First, there is no preprocessing stage, no manifest, and no cache to invalidate when the excerpt length or sample rate changes. Second, memory is independent of corpus size — a 26.4-hour, 10.5 GB source file costs nothing to hold because it is never held. Third, the excerpt distribution is continuous rather than quantised to cache-entry boundaries, so the model sees a different alignment of musical structure on every epoch.

The cost is negligible: a random seek-and-decode of a 24s excerpt from a 10.5 GB FLAC takes 31 ms, and two dataloader workers hide it entirely behind GPU work.

6 Fitting a full finetune on one GPU

Training all 1.453 B transformer parameters naively is out of reach: float32 weights, float32 gradients and float32 Adam moments alone require $1.453\text{B} \times 16\text{B} = 23.2\text{GB}$, before activations, before the autoencoder, and before the text encoder. Three choices bring it inside 24 GB.

1. Eight-bit Adam moments. The optimiser state drops from 11.6 GB to 2.9 GB. Master weights and gradients stay in float32, so the update itself is unaffected in precision where it matters.
2. The frozen autoencoder in bfloat16. It never receives gradients, so half precision costs nothing and saves 1.7 GB.
3. Gradient checkpointing throughout the transformer.

Releasing the text encoder after its single use recovers a further 1.2 GB. The resulting measured peak is 15.7 GiB, leaving comfortable headroom for the sampling and decoding done during periodic demos.

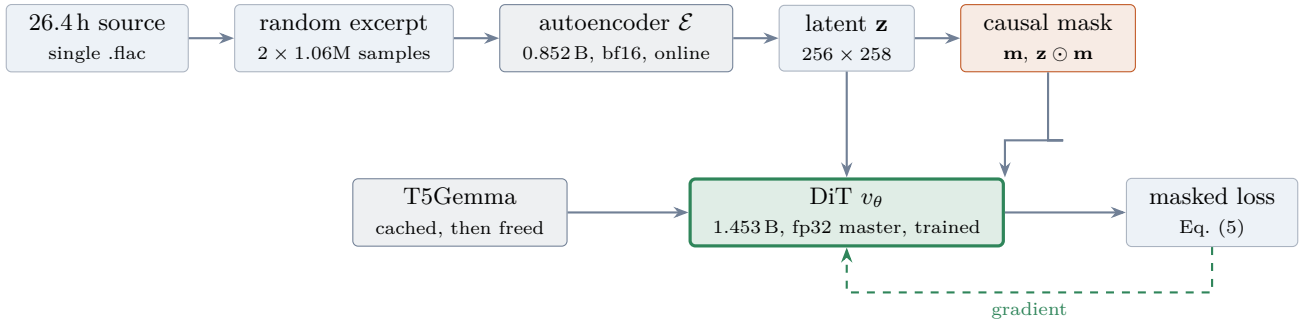


Figure 5: System overview. Grey blocks are frozen, green is trained. Nothing is precomputed: each step reads fresh audio off disk and encodes it in the same step it is used. The text encoder is evaluated once and then released, since its output is constant for a run.

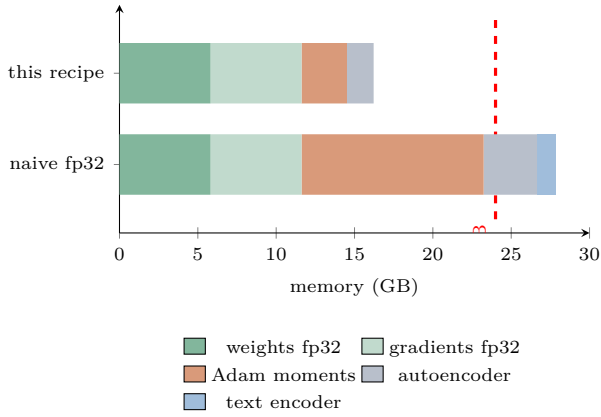


Figure 6: Memory budget. Eight-bit optimiser moments, a bfloat16 frozen autoencoder and releasing the text encoder move a 27.9 GB configuration to a measured 15.7 GiB peak. Activations under gradient checkpointing are small and not shown separately.

6.1 Checkpoint transport

The released checkpoint is 9.2 GB of float32 across 997 tensors (1.453 B transformer, 0.852 B autoencoder). Since training holds float32 master weights in memory regardless, storing bfloat16 on disk costs a single rounding at load. We therefore convert during download, streaming tensors in file order and writing a bfloat16 safetensors file directly, so the float32 artefact is never materialised. The stored model is 4.6 GB.

7 Cost structure

The autoencoder dominates, costing roughly nine times the transformer step it feeds. This is the honest price of online encoding, and it follows directly from the architecture: a 0.852 B encoder that must process 8085 patch tokens to emit 258 latent frames does substantially more work than a 1.453 B transformer operating on those 258 frames.

We note that this cost is a choice, not a constraint.

Stage	Time (s)
Excerpt read and decode (2 workers)	0.03
Autoencoder encode (online)	4.00
DiT forward + backward	0.45
Optimiser step	< 0.01
Total per step	4.70

Table 1: Measured per-step cost, RTX 3090, batch 4, 24s excerpts.

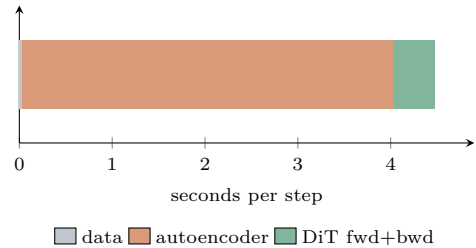


Figure 7: Where the step goes. Online encoding is 85% of wall clock. The trade is deliberate: it buys a pipeline with no preprocessing stage and no cache, at roughly 6× the per-step cost of training from precomputed latents.

The training loop consumes $\mathbf{z}$ and is indifferent to its provenance; substituting a precomputed latent cache recovers the 4s and changes nothing else. We prefer the online path for single-corpus finetunes, where setup cost dominates total cost.

8 Evaluation protocol

Rectified-flow training loss at a random timestep is a high-variance estimator of anything one would want to monitor. Figure 8 makes the point concretely: over the first 370 steps of a real run, the per-step loss oscillates between 0.63 and 1.34 and supports no conclusion about whether the model is improving.

We therefore fix an evaluation slice at startup and never resample it: four excerpts drawn from fixed off-

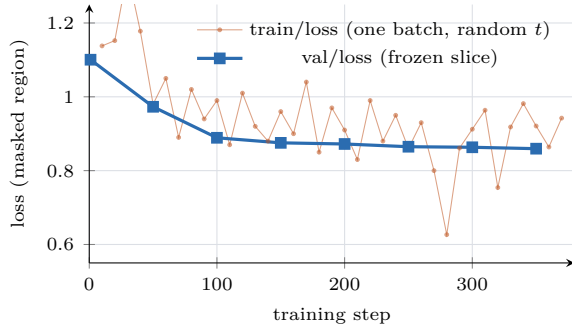


Figure 8: Training loss is not a learning signal here. The per-step loss (orange) spans 0.63–1.34 with no discernible trend, because each point is a single batch at a single randomly drawn timestep and the difficulty of Eq. (3) varies sharply with t . Holding excerpts, masks, timesteps and noise fixed (blue) yields a monotone curve from the same run.

Step	val/loss	Δ	train/loss range
1	1.1005	—	—
50	0.9730	−0.1275	0.98–1.34
100	0.8890	−0.0840	0.87–1.05
150	0.8753	−0.0137	0.85–1.04
200	0.8722	−0.0031	0.83–0.99
250	0.8648	−0.0074	0.86–0.99
300	0.8633	−0.0015	0.63–0.96
350	0.8595	−0.0038	0.86–0.98

Table 2: The frozen slice decreases monotonically while the per-step loss range is wide and overlapping throughout.

sets, each with a fixed causal mask at a different prefix fraction (0.25 to 0.75), crossed with four fixed timesteps at $\{0.125, 0.375, 0.625, 0.875\}$, each with fixed noise. The reported value is the mean masked error over these sixteen configurations. Every source of variance except the weights is frozen, so successive values are directly comparable, and the cost is one second per evaluation.

Alongside the scalar we periodically sample a continuation of one fixed reference excerpt with the distilled eight-step ping-pong sampler and log the audio. Fixing the reference is what makes successive clips comparable by ear; sampling a fresh excerpt each time confounds model progress with excerpt difficulty.

9 Recipe summary

Table 3 collects every value needed to reproduce the method. Only three entries differ from the base model’s own pretraining configuration: the mask-type probabilities, which are reweighted toward Causal; the optimiser, which uses eight-bit moments in place of the original MuonAdamW; and the excerpt length, which is shortened to the bottom of the model’s supported range. Ev-

Base model	Stable Audio 3 Medium
Trained	DiT, 1.453 B, all parameters
Frozen	autoencoder 0.852 B, text encoder
Latent	256 ch @ 10.77 Hz (/4096)
Excerpt	24 s = 258 latent frames
Batch	4
Objective	rectified flow, masked to $\mathbf{m}=0$
Mask types	$[0, 0.1, 0.9]$ spans/full/causal
Timesteps	truncated logistic-normal, shifted
CFG dropout	0.1
Optimiser	AdamW, 8-bit moments
LR	10^{-5} , 100-step linear warmup
Betas / WD	(0.9, 0.95) / 0.01
Grad clip	1.0
Precision	fp32 master, bf16 autocast
Peak memory	15.7 GiB
Throughput	4.7 s/step (RTX 3090)

Table 3: Complete hyperparameter set.

Model	Envelope dist.	Level vs. truth
Pretrained	0.279	1.28×
Finetuned, 2000 steps	0.296	1.28×
Finetuned, 6000 steps	0.349	0.72×
Blend $\alpha = 0.25$	0.296	1.28×
Blend $\alpha = 0.50$	0.324	1.08×

Table 4: The finetune does not improve on the pretrained checkpoint. Envelope distance degrades monotonically as the finetuned weights are blended in.

everything else — the noise schedule, the timestep sampler, the guidance dropout rate — is inherited unchanged, which is the point.

10 Results

We evaluate by outpainting: 32s of real audio is given as context and the model generates the following 158s, at the eight sampler steps the checkpoint is distilled for. Quality is the mean absolute difference between the generated region’s 24-band log-spectral envelope and that of the true continuation, plus the ratio of generated RMS to the true continuation’s RMS. Five contexts are drawn from widely separated points of a 26.4 h source.

10.1 A negative result, and its mechanism

The finetune systematically reduces output energy. Across five contexts the generated region falls to 0.55–0.72× the true level, from early in training, and the effect grows with how much audio the model must invent. Three ablations locate the cause:

1. Not dosage. Stopping at 2000 steps instead of 6000 reduces but does not remove it (1/5 contexts beat the baseline instead of 0/5).

2. Not the gates. Freezing every 1-D parameter — the RMSNorm gains and adaLN modulation vectors that globally rescale each block — leaves the level ratio unchanged at $0.58\times$ against $0.59\times$. The drift originates in the weight matrices, so it is the velocity field itself, not a global gain.
3. Not recoverable by interpolation. Blending back toward the base improves the level monotonically but never beats the base on envelope (Table 4).

A fourth observation identifies the likely cause. The released checkpoint is adversarially post-trained for few-step sampling. The pretrained model degrades gracefully as sampler steps increase ($0.412 \rightarrow 0.577$ from 8 to 64 steps); the finetuned model collapses ($0.221 \rightarrow 2.908$, with output falling to 0.017 RMS and the spectral centroid rising to 9.2 kHz, i.e. hiss). Plain flow-matching MSE pulls the velocity field toward the conditional mean, which erodes exactly the property adversarial post-training installed. Notably, Stability ships a LoRA finetuning script for this model and not a full-finetune one.

We therefore report this as a negative result: the recipe of §4.1–§8 is correct with respect to the reference implementation, trains stably, and reduces its training objective, yet does not improve perceptual quality on an adversarially distilled checkpoint. Recovering the missing adversarial term, or restricting the update to a low-rank adapter, are the two obvious next steps.

11 Discussion

The headline lesson is negative and worth stating plainly: a finetune can be faithful to its reference implementation, train stably, and lower its loss while making the model worse. Our validation loss fell from 1.31 to 1.01 and the context-reconstruction term fell twenty-fold, over exactly the interval in which output energy drifted to half the source’s. Any evaluation that had stopped at the loss curve would have reported success.

The general lesson is that capability is often already latent in a pretrained checkpoint, distributed across a conditioning interface that was trained for a broader task than the one at hand. Stable Audio 3 was trained to inpaint; outpainting is the causal corner of that distribution. Specialising to it is a change of sampling weights, not a change of model. The practical payoff is disproportionate: no zero-initialised branch to warm up, no train/inference conditioning mismatch, and a finetune whose step zero is exactly the released model.

Two limitations are worth naming. Online encoding costs 85% of wall-clock time, which is the right trade for single-corpus finetunes and the wrong one for large multi-corpus runs. And a full finetune on one domain will move the model away from its general prior; the residual Full-mask fraction slows this but does not eliminate it.

12 Conclusion

We presented a minimal recipe for specialising a latent rectified-flow audio diffusion model to outpainting, with no architectural modification, on a single consumer GPU. The method reduces to reweighting a pretrained mask distribution, masking the loss to the generated region, and choosing precisions that let 1.453 B parameters train in 24 GB. We additionally argued that per-step loss is an inadequate monitor for this objective and gave a frozen-slice protocol that is not. Code is available at <https://github.com/mrcolo/stems-loop>.